

Assignment -4

Name : ISHA MADEEHA

HALL TICKETNO:2303A51017

Batch-01

Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques, Chain of Thoughts, Prompt Chaining, Directional stimulus prompting

Lab Objectives:

- To explore and apply different levels of prompt examples in AI-assisted code generation.
- To understand how zero-shot, one-shot, and few-shot prompting affect AI output quality.
- To evaluate the impact of context richness and example quantity on AI performance.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use zero-shot prompting to instruct AI with minimal context.
- Use one-shot prompting with a single example to guide AI code generation.
- Apply few-shot prompting using multiple examples to improve AI responses.
- Compare AI outputs across the three prompting strategies.

Q1. Zero-Shot Prompting (Basic Lab Task)

Task:

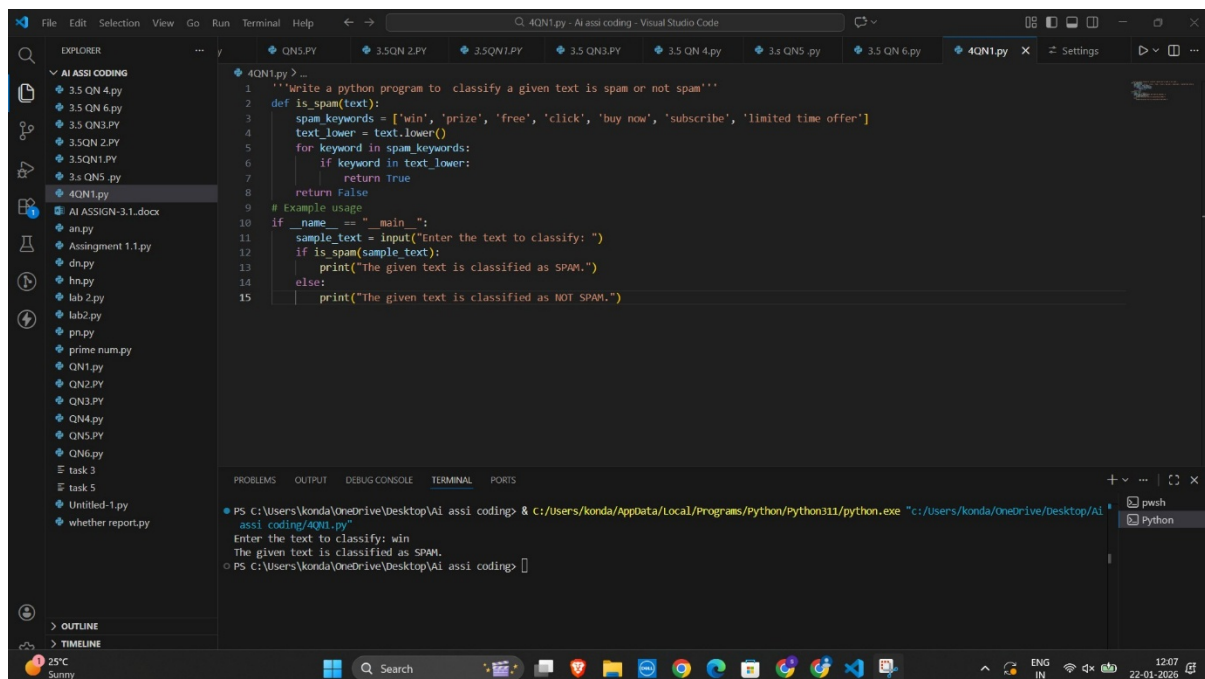
Write a Python function that classifies a given text as Spam or Not Spam using zero-shot prompting.

Steps:

1. Construct a prompt without any examples.
2. Clearly specify the output labels.
3. Display only the predicted label.

Input:

"Congratulations! You have won a free lottery ticket."



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left lists files under 'AI ASSI CODING', including '4QN1.py'. The main editor displays the code for '4QN1.py', which defines a function 'is_spam(text)' that checks for keywords in a list. The code also includes a main block that prompts the user for text and prints the classification result. The Terminal panel at the bottom shows the execution of the script, where the input 'win' results in the output 'The given text is classified as SPAM.'

```
1 '''write a python program to classify a given text is spam or not spam'''
2 def is_spam(text):
3     spam_keywords = ['win', 'prize', 'free', 'click', 'buy now', 'subscribe', 'limited time offer']
4     text_lower = text.lower()
5     for keyword in spam_keywords:
6         if keyword in text_lower:
7             return True
8     return False
9 # Example usage
10 if __name__ == "__main__":
11     sample_text = input("Enter the text to classify: ")
12     if is_spam(sample_text):
13         print("The given text is classified as SPAM.")
14     else:
15         print("The given text is classified as NOT SPAM.")
```

Terminal Output:

```
PS C:\Users\konda\OneDrive\Desktop\AI assi coding> & c:/Users/konda/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/konda/OneDrive/Desktop/AI assi coding/4QN1.py"
Enter the text to classify: win
The given text is classified as SPAM.
PS C:\Users\konda\OneDrive\Desktop\AI assi coding>
```

Q2. One-Shot Prompting (Emotion detection)

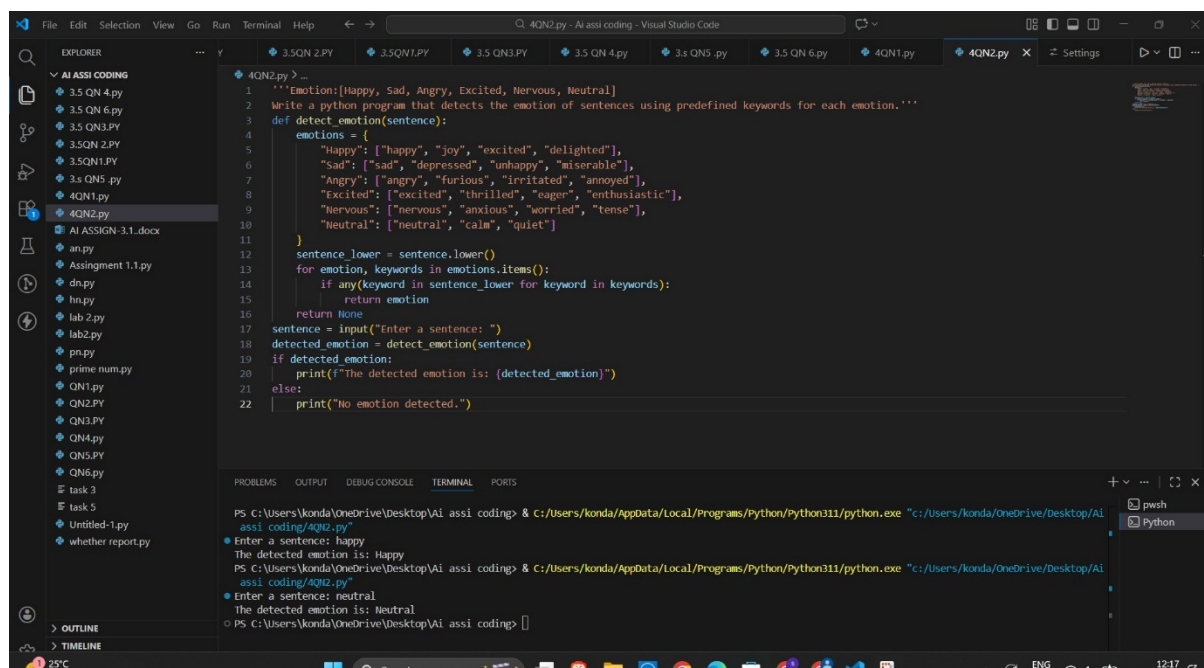
Task:

Write a Python program that detects the emotion of a sentence using one-shot prompting.

Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']

Steps:

1. Provide one labeled example inside the prompt.
2. Take a sentence as input.
3. Print the predicted emotion



```
4QN2.py > ...
1 '''Emotion:[Happy, Sad, Angry, Excited, Nervous, Neutral]
2 Write a python program that detects the emotion of sentences using predefined keywords for each emotion.'''
3 def detect_emotion(sentence):
4     emotions = {
5         "Happy": ["happy", "joy", "excited", "delighted"],
6         "Sad": ["sad", "depressed", "unhappy", "miserable"],
7         "Angry": ["angry", "furious", "irritated", "annoyed"],
8         "Excited": ["excited", "thrilled", "eager", "enthusiastic"],
9         "Nervous": ["nervous", "anxious", "worried", "tense"],
10        "Neutral": ["neutral", "calm", "quiet"]
11    }
12    sentence_lower = sentence.lower()
13    for emotion, keywords in emotions.items():
14        if any(keyword in sentence_lower for keyword in keywords):
15            return emotion
16    return None
17 sentence = input("Enter a sentence: ")
18 detected_emotion = detect_emotion(sentence)
19 if detected_emotion:
20     print(f"The detected emotion is: {detected_emotion}")
21 else:
22     print("No emotion detected.")
```

Terminal Output:

```
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding> & c:/Users/konda/AppData/Local/Programs/Python/python311/python.exe "c:/Users/konda/OneDrive/Desktop/Ai assi coding/4QN2.py"
Enter a sentence: happy
The detected emotion is: Happy
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding> & c:/Users/konda/AppData/Local/Programs/Python/python311/python.exe "c:/Users/konda/OneDrive/Desktop/Ai assi coding/4QN2.py"
Enter a sentence: neutral
The detected emotion is: Neutral
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding>
```

Q3. Few-Shot Prompting (Student Grading Based on Marks)

Task:

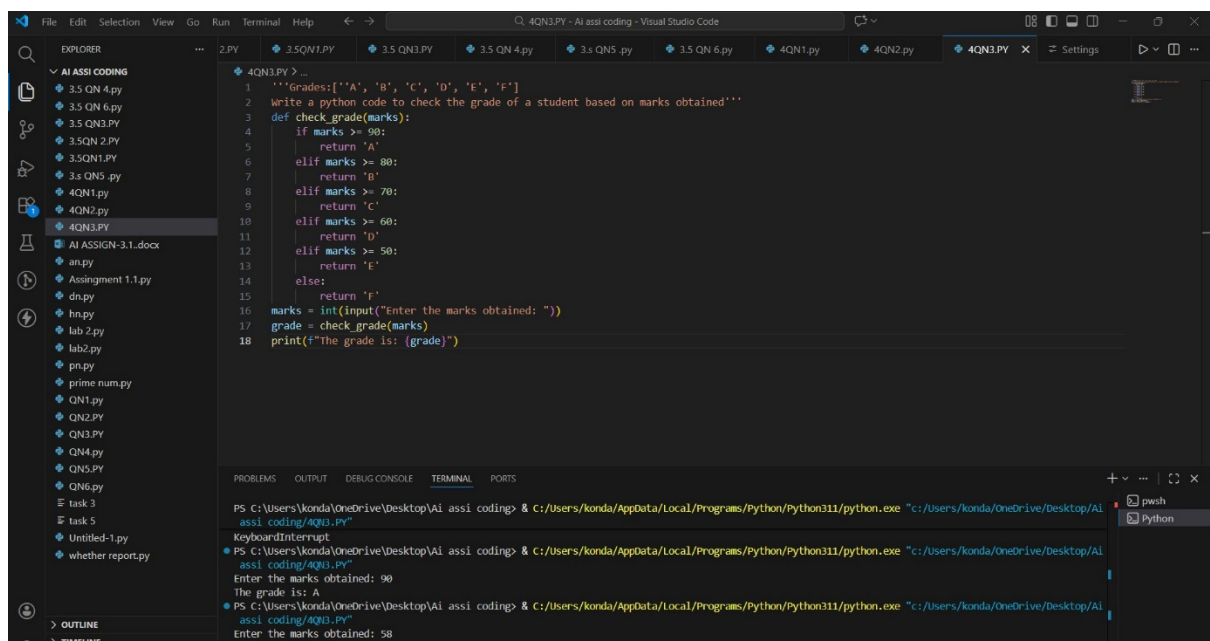
Write a Python program that predicts a student's grade based on marks using few-shot prompting.

Grades:

['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

- 90–100 → A
- 80–89 → B
- 70–79 → C
- 60–69 → D
- Below 60 → F



The screenshot shows a Visual Studio Code window with a Python file named `4QN3.PY` open. The code defines a function `check_grade(marks)` that returns a grade based on the input marks. The grading criteria are: 90-100 for 'A', 80-89 for 'B', 70-79 for 'C', 60-69 for 'D', and below 60 for 'F'. The program prompts the user to enter marks and prints the resulting grade.

```
1 '''Grades:['A', 'B', 'C', 'D', 'E', 'F']
2 Write a python code to check the grade of a student based on marks obtained'''
3 def check_grade(marks):
4     if marks >= 90:
5         return 'A'
6     elif marks >= 80:
7         return 'B'
8     elif marks >= 70:
9         return 'C'
10    elif marks >= 60:
11        return 'D'
12    elif marks >= 50:
13        return 'E'
14    else:
15        return 'F'
16 marks = int(input("Enter the marks obtained: "))
17 grade = check_grade(marks)
18 print(f"The grade is: {grade}")
```

The terminal output shows the program being executed. The user enters 90, and the program outputs "The grade is: A". The user then enters 58, and the program outputs "The grade is: F".

```
PS C:\Users\konda\OneDrive\Desktop\AI assi coding> & C:/Users/konda/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/konda/OneDrive/Desktop/AI
assi coding/4QN3.PY"
KeyboardInterrupt
PS C:\Users\konda\OneDrive\Desktop\AI assi coding> & C:/Users/konda/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/konda/OneDrive/Desktop/AI
assi coding/4QN3.PY"
Enter the marks obtained: 90
The grade is: A
PS C:\Users\konda\OneDrive\Desktop\AI assi coding> & C:/Users/konda/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/konda/OneDrive/Desktop/AI
assi coding/4QN3.PY"
Enter the marks obtained: 58
The grade is: F
```

Q4. Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task:

Write a Python program that predicts a person's Indian Zodiac sign (Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model): The Indian Zodiac cycle starts in March with Mesha and follows this order:

March → Mesha

April → Vrishabha

May → Mithuna

June → Karka

July → Simha

August → Kanya

September → Tula

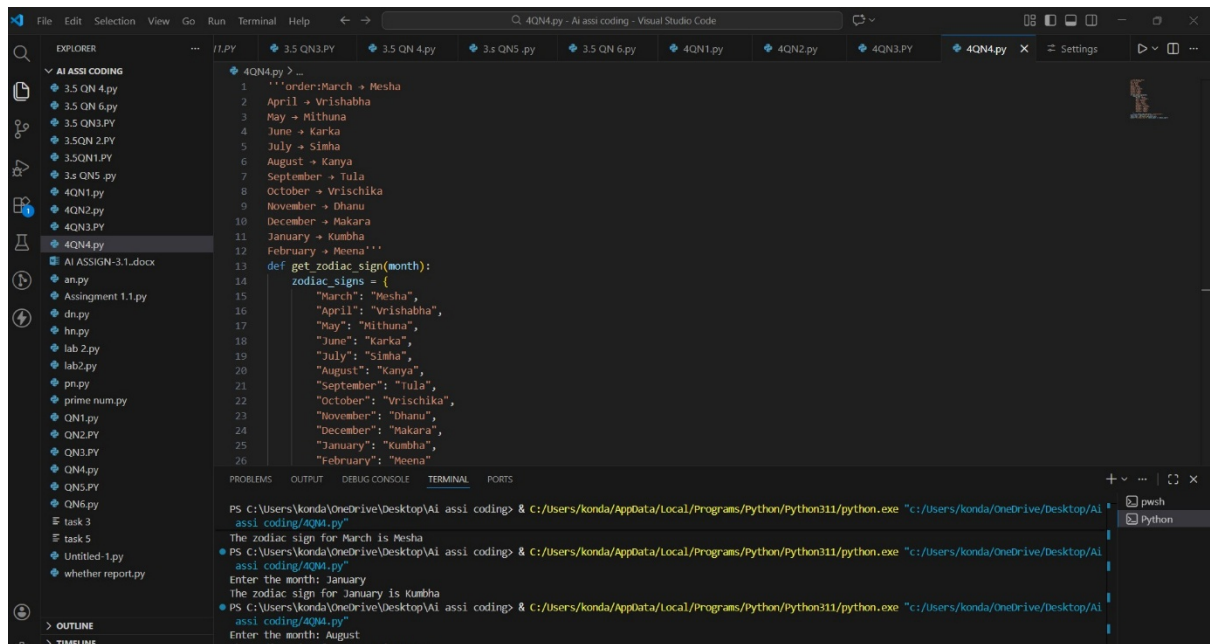
October → Vrischika

November → Dhanu

December → Makara

January → Kumbha

February → Meena



Q5. Result Analysis Based on Marks

Task: Write a Python program that determines whether a student

Passes or Fails based on marks using Chain-of-Thought (CoT)

prompting.

Result Categories:

['Pass', 'Fail']

```
1 '''Write a python program to determine whether a student passes or fails
2 creating a variable 'marks' to store the marks obtained by the student. The passing criteria is 40 marks.
3 If the marks are 40 or more, print 'pass', otherwise print 'fail'.
4 result: pass if marks >= 40 else fail'''
5 marks = int(input("Enter the marks obtained by the student: "))
6 if marks >= 40:
7     print("pass")
8 else:
9     print("fail")
```

```
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding> & C:\Users\konda\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/konda/OneDrive/Desktop/Ai
assi coding/4QN 5.py"
Enter the marks obtained by the student: 98
pass
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding> & C:\Users\konda\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/konda/OneDrive/Desktop/Ai
assi coding/4QN 5.py"
Enter the marks obtained by the student: 45
pass
```

Q6 Voting Eligibility Check (Chain-of-Thought Prompting)

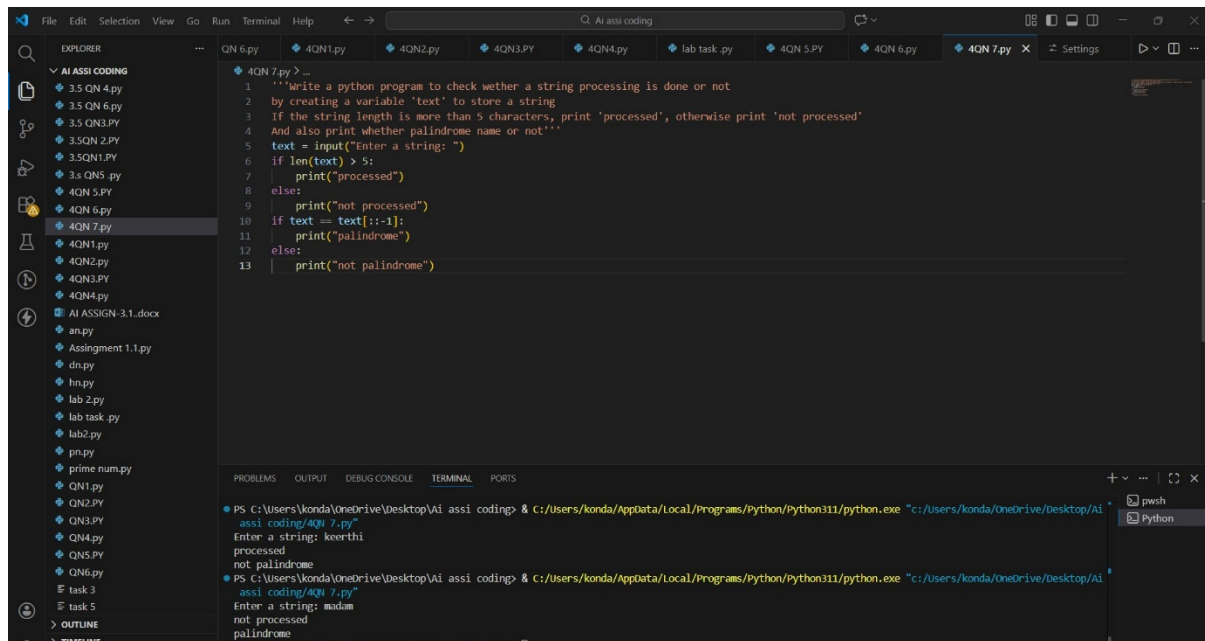
Task: Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting.

```
1 '''write a python program to check voting eligibility
2 by creating a variable 'age' to store the age of the person.
3 If the age is 18 or more, print 'eligible to vote', otherwise print 'not eligible to vote'''
4 age = int(input("Enter the age of the person: "))
5 if age >= 18:
6     print("eligible to vote")
7 else:
8     print("not eligible to vote")
9
```

```
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding> & C:\Users\konda\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/konda/OneDrive/Desktop/Ai
assi coding/4QN 6.py"
Enter the age of the person: 20
eligible to vote
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding> & C:\Users\konda\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/konda/OneDrive/Desktop/Ai
assi coding/4QN 6.py"
Enter the age of the person: 16
not eligible to vote
PS C:\Users\konda\OneDrive\Desktop\Ai assi coding>
```

Q7 Prompt Chaining (String Processing – Palindrome Names)

Task: Write a Python program that uses the prompt chaining technique to identify palindrome names from a list of student names.



```
File Edit Selection View Go Run Terminal Help
4QN 7.py > -
1 '''write a python program to check wether a string processing is done or not
2 by creating a variable 'text' to store a string
3 If the string length is more than 5 characters, print 'processed', otherwise print 'not processed'
4 And also print whether palindrome name or not'''
5 text = input("Enter a string: ")
6 if len(text) > 5:
7     print("processed")
8 else:
9     print("not processed")
10 if text == text[::-1]:
11     print("palindrome")
12 else:
13     print("not palindrome")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\konda\OneDrive\Desktop\ai assi coding> & C:/Users/konda/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/konda/OneDrive/Desktop/ai
assi coding/4QN 7.py"
Enter a string: keerthi
processed
not palindrome
PS C:\Users\konda\OneDrive\Desktop\ai assi coding> & C:/Users/konda/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/konda/OneDrive/Desktop/ai
assi coding/4QN 7.py"
Enter a string: madam
not processed
palindrome
```

Q8 Prompt Chaining (String Processing – Word Length

Analysis)

Task: Write a Python program that uses prompt chaining to analyze a list of words. In the first prompt, generate a list of words. In the second prompt, traverse the list and calculate the length of each word. In the third prompt, use the output of the previous step to determine whether each word is Short (length less than 5) or Long (length greater than or equal to 5), and display the result for each word

